API SECURITY RISK ANALYSIS REPORT

Target API: JSONPlaceholder (jsonplaceholder.typicode.com)

Assessment Type: Read-Only Security Audit

Date: April 2026

Prepared by: Titus Joshua

Tools Used: Postman (Lightweight Client), Browser DevTools

Classification: *Educational / Public API Audit*

1. Executive Summary

This report presents the findings of a read-only API security risk analysis conducted against JSONPlaceholder, a publicly available REST API used for testing and prototyping. The assessment was performed in accordance with OWASP API Security Top 10 guidelines and followed an ethical, non-exploitative methodology.

Five security risks were identified across authentication, authorization, data exposure, and response header controls. Of these, two are rated HIGH severity, two MEDIUM, and one LOW.

Risk Summary

Risk ID	Risk Title	Severity	OWASP Ref
R-01	Unauthenticated Access to User Data	HIGH	API1:2023
R-02	Broken Object Level Authorization (IDOR)	HIGH	API1:2023
R-03	Unauthenticated Write Access	MEDIUM	API5:2023
R-04	Missing Security Response Headers	MEDIUM	API7:2023
R-05	Technology Stack Disclosure	LOW	API8:2023

2. Scope & Methodology

2.1 Scope

The following endpoints were tested during this assessment:

- GET https://jsonplaceholder.typicode.com/users
- GET https://jsonplaceholder.typicode.com/users/{id}
- GET https://jsonplaceholder.typicode.com/posts
- POST https://jsonplaceholder.typicode.com/posts
- Response Headers inspection on all above endpoints

2.2 Methodology

- Documentation review of the target API
- Endpoint enumeration using Postman Lightweight Client
- Authentication and authorization testing
- Response data analysis for excessive exposure
- Security header inspection
- Risk classification using OWASP API Security Top 10 (2023)

2.3 Tools Used

- Postman (Lightweight API Client) → endpoint testing and response inspection
- Browser DevTools → header analysis
- OWASP API Security Top 10 (2023) → risk classification framework

3. Detailed Findings

R-01 | Unauthenticated Access to User Data: HIGH

Endpoint	GET /users
Observation	The /users endpoint returns all user records including name, email, address, phone, GPS coordinates and company - with no authentication required.
Business Impact	An unauthenticated attacker can harvest all user PII in a single request. In production this constitutes a data breach triggering GDPR obligations.
Evidence	HTTP 200 OK returned with 10 full user objects. No Authorization header required. See screenshot below.
Remediation	Implement JWT or OAuth 2.0 bearer token authentication on all user endpoints. Apply data minimization - return only fields needed by the caller.

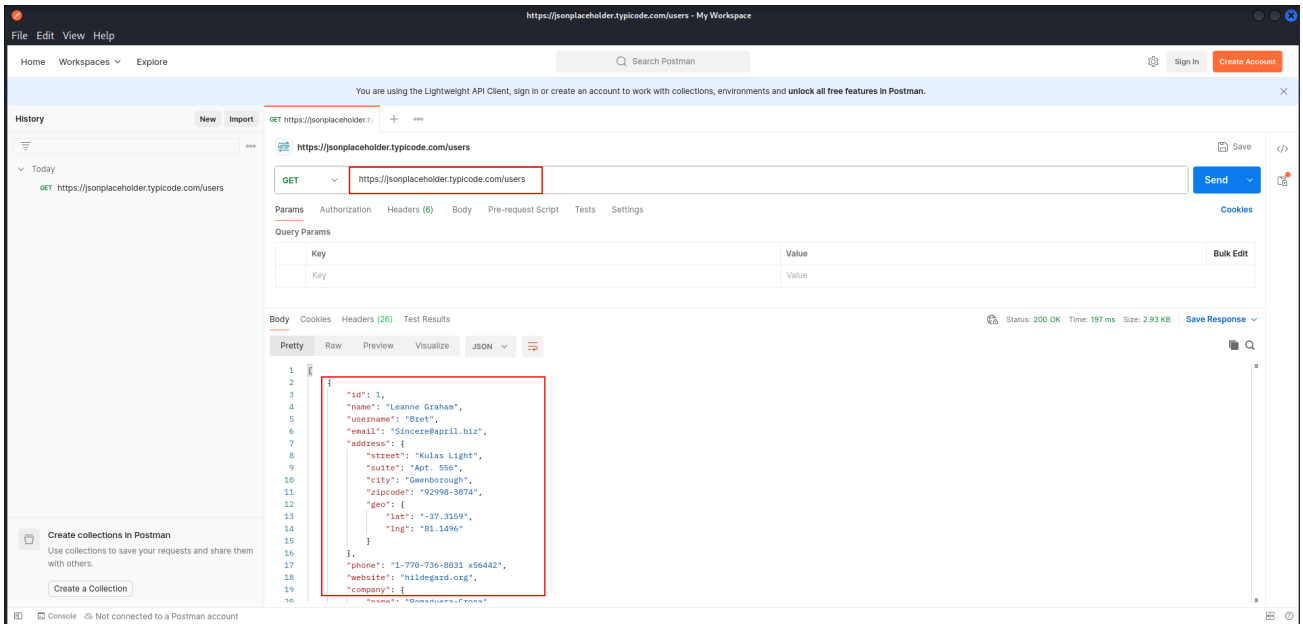

Figure 1: GET /users returns full PII for all users with no authentication (HTTP 200 OK)

R-02 | Broken Object Level Authorization (IDOR): HIGH

Endpoint	GET /users/1, GET /users/2
Observation	Incrementing the user ID in the URL from /users/1 to /users/2 returns a completely different user's profile with no ownership check.
Business Impact	Attackers can enumerate all accounts by iterating IDs, enabling targeted phishing, social engineering, or identity theft at scale.
Evidence	GET /users/2 returns Ervin Howell's full profile with HTTP 200 OK and no authorization token. See screenshot below.
Remediation	Implement object-level authorization checks per request. Use non-sequential UUIDs instead of integer IDs. Alert on enumeration patterns.

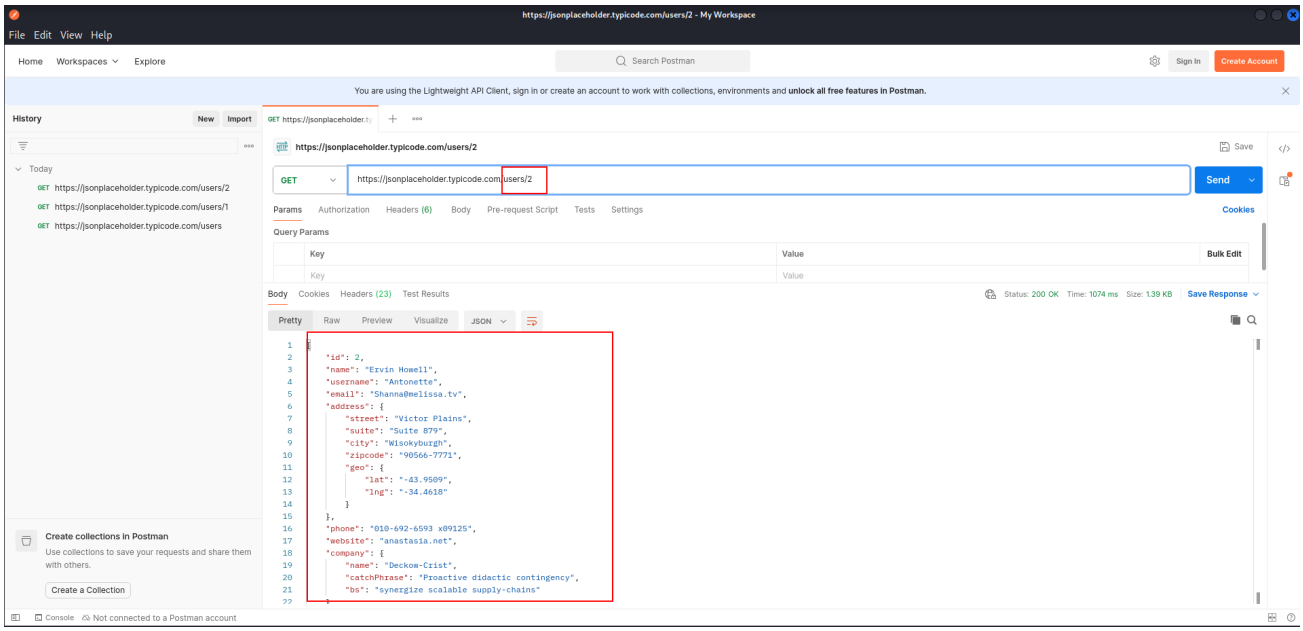

Figure 2: GET /users/2 returns a different user's full profile - classic IDOR, no auth required

R-03 | Unauthenticated Write Access: MEDIUM

Endpoint	POST /posts
Observation	A POST request with JSON body was accepted and returned HTTP 201 Created without any authentication token. Resource created with id: 101.
Business Impact	Unauthenticated write access allows content injection, data corruption, and API abuse for spam or automated attacks.
Evidence	POST /posts with body {title, body, userId} returned HTTP 201 Created with id:101. No Authorization header was sent. See screenshots below.
Remediation	Require authentication tokens for all write operations. Validate userId in request body matches the authenticated user's token claims.

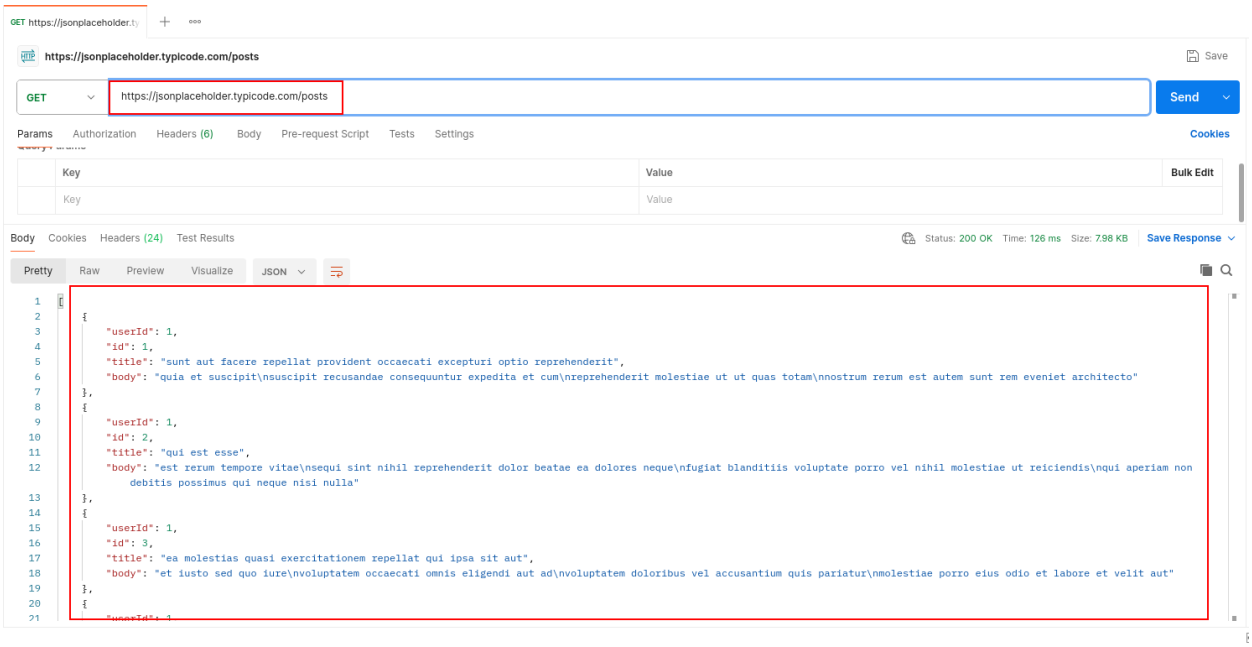

Figure 3: GET /posts - all posts returned publicly with no authentication

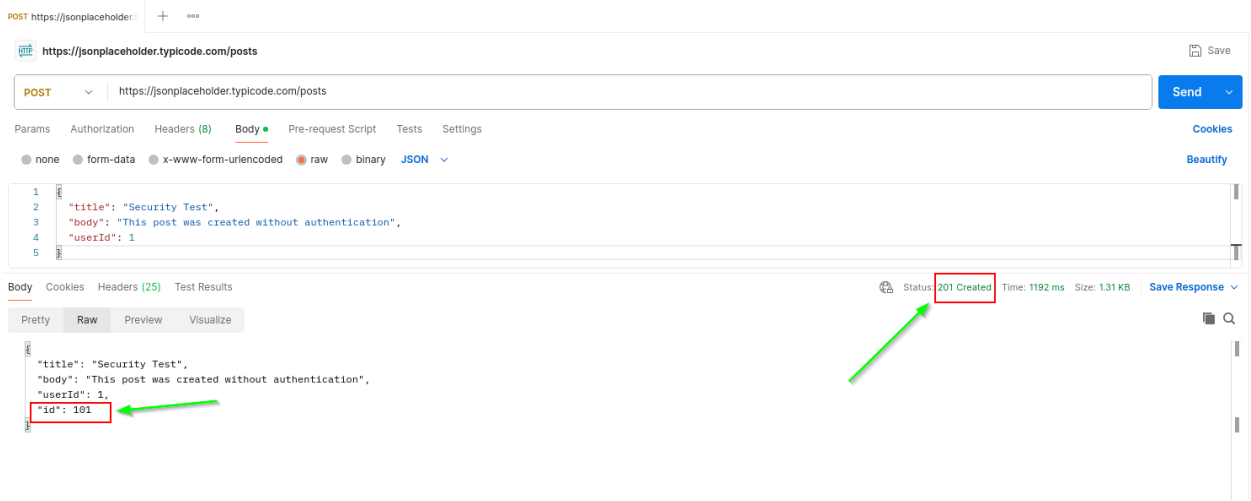

Figure 4: POST /posts returns HTTP 201 Created with no authentication token - resource created as id:101

R-04 | Missing Security Response Headers: MEDIUM

Endpoint	All tested endpoints
Observation	The following critical headers are absent: Content-Security-Policy, X-Frame-Options, Strict-Transport-Security. Only x-content-type-options: nosniff is present.
Business Impact	Missing CSP enables XSS attacks. Missing X-Frame-Options allows clickjacking. Missing HSTS allows man-in-the-middle attacks over HTTP.
Evidence	Headers tab inspection confirmed absence of CSP, X-Frame-Options, and HSTS across all responses. See screenshot below.
Remediation	Add at API gateway level: Content-Security-Policy: default-src 'self', X-Frame-Options: DENY, Strict-Transport-Security: max-age=31536000.

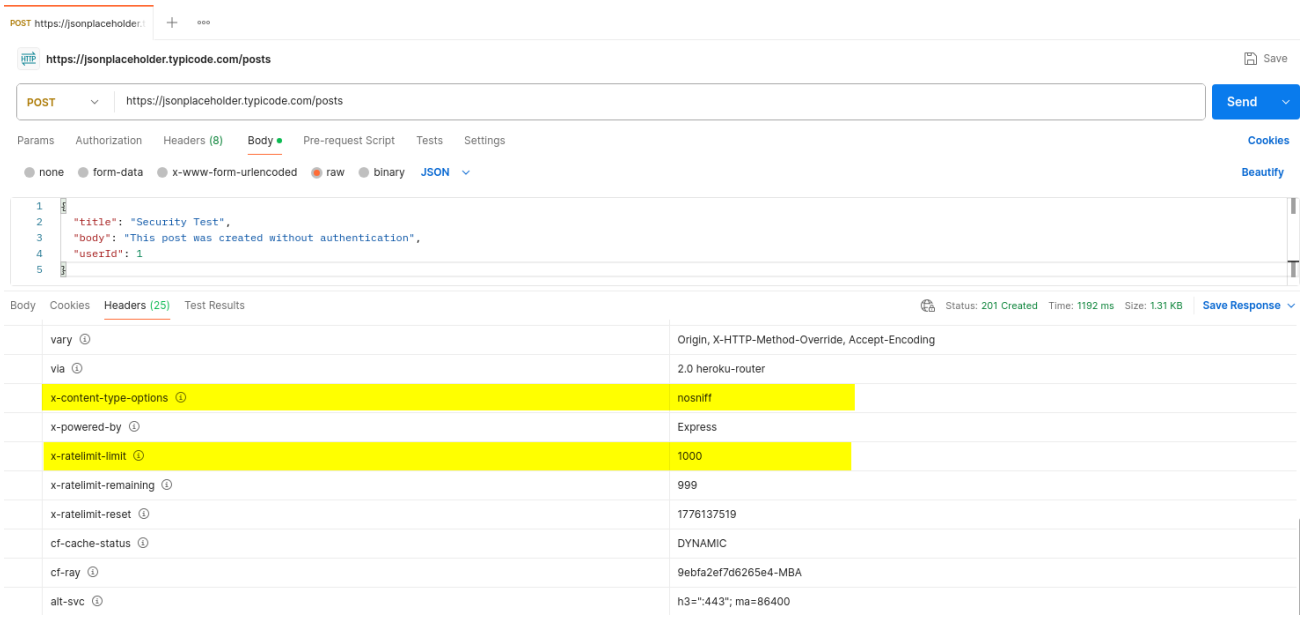

Figure 5: Response headers CSP, X-Frame-Options, and HSTS are all absent. x-powered-by exposes Express framework.

R-05 | Technology Stack Disclosure: LOW

Endpoint	All tested endpoints
Observation	Response header x-powered-by: Express reveals the API is built on Express.js. The via: 2.0 heroku-router header discloses the hosting platform.
Business Impact	Exposing the tech stack assists attackers in targeting known framework-specific vulnerabilities, lowering the effort for targeted attacks.
Evidence	Headers confirm: x-powered-by: Express and via: 2.0 heroku-router visible in all API responses (see Figure 5).
Remediation	Suppress in Express.js with: app.disable('x-powered-by'). Configure the reverse proxy to omit the via header.

4. Full Risk Matrix

ID	Risk Title	Severity	Business Impact	Remediation
R-01	Unauthenticated Access to User Data	HIGH	Full PII exposure without login. GDPR breach risk.	Implement JWT/OAuth2 on all user endpoints
R-02	Broken Object Level Authorization	HIGH	Any user's data accessible by ID enumeration.	Object-level auth checks + UUID IDs
R-03	Unauthenticated Write Access	MEDIUM	Data injection and content abuse without auth.	Require auth tokens for all write operations
R-04	Missing Security Headers	MEDIUM	XSS, clickjacking, and MITM attack exposure.	Add CSP, X-Frame-Options, HSTS headers
R-05	Technology Stack Disclosure	LOW	Framework fingerprinting aids targeted attacks.	Disable x-powered-by header

5. Recommendations Summary

Priority 1: Immediate (High Severity)

- Deploy authentication middleware (JWT or OAuth 2.0) on all endpoints before any production use
- Implement object-level authorization to prevent IDOR validate resource ownership per request

Priority 2: Short Term (Medium Severity)

- Enforce authentication on all write operations (POST, PUT, DELETE)
- Configure security headers at the API gateway: CSP, X-Frame-Options, HSTS

Priority 3: Ongoing (Low Severity)

- Suppress technology disclosure headers (x-powered-by, via)
- Conduct regular API security reviews aligned to OWASP API Security Top 10
- Implement API activity logging and anomaly detection for enumeration attempts

6. Disclaimer

This assessment was conducted solely for educational purposes against a public demo API (JSONPlaceholder) that is explicitly provided for testing and learning. No production systems were targeted. No data was exfiltrated, modified, or deleted. All testing followed ethical guidelines and remained within the defined scope.